

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report

Of

Cloud Computing and DevOps Lab [CSE 3130]

WARDEN

A Verifiable Kubernetes Delivery Platform

SUBMITTED BY

Student	Registration number
Abhyuday Gupta	240905576
Aditya Chetan Thakur	240905588
Sankalp Arya	240905578
Shaurya Jain	240905598

Under the Guidance of

Mohandas Shenoy

School of Computer Engineering

Manipal Institute of Technology, Manipal

Academic Year 2026–27

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

CERTIFICATE

This is to certify that the mini project report entitled "Warden: A Verifiable Kubernetes Delivery Platform" is submitted by Abhyuday Gupta (240905576), Aditya Chetan Thakur (240905588), Sankalp Arya (240905578), and Shaurya Jain (240905598) for the Cloud Computing and DevOps Lab [CSE 3130], School of Computer Engineering, Manipal Institute of Technology, during the academic year 2026–27.

The report describes the work and results presented by the team. The signatures below are reserved for examination and approval.

Project guide	**Examiner**
Signature: _____	Signature: _____
Mohandas Shenoy	Name: _____
Date: _____	Date: _____

TABLE OF CONTENTS

CHAPTER 1: INTRODUCTION **4**

1.1 Context and 1.2 Objectives 4

1.3 Cloud services and scope 5

1.4 Research foundations and 1.5 Contribution 5

CHAPTER 2: METHODOLOGY **6**

2.1 Infrastructure and 2.2 Trusted build 6

2.3 Promotion, admission, and trust boundaries 7

2.4 Database and 2.5 Progressive delivery 8

2.6 Evaluation design 8

CHAPTER 3: RESULT ANALYSIS **9**

3.1 Experiment summary 9

3.2 Canary behavior 10

3.3 Delivery and rebuild evidence 11

3.4 Admission and database limits 11

3.5 Comparison and 3.6 Engineering findings 13

CHAPTER 4: CONCLUSION **14**

TEAM RESPONSIBILITIES 15

REFERENCES **16**

CHAPTER 1: INTRODUCTION

1.1 Context

Warden brings service creation, software supply-chain checks, GitOps delivery, and automatic canary decisions into one Kubernetes platform. Five complete local trials created and deployed a service in a median of 8.6 minutes with no manual steps. In a separate 20-release canary experiment, all ten faulty releases aborted and all ten healthy releases promoted. Five successful Azure rebuilds demonstrated that the platform could be recreated from its recorded configuration.

These results address a practical engineering problem: a successful build does not by itself establish what will run in a cluster or how a faulty release will be contained. Warden links an image digest to its build evidence, promotes that digest through a controlled Git change, and evaluates the release under traffic. The four sample services, books, loans, members, and notify, provide repeatable workloads for testing this delivery path.

The platform covers service creation, build and scanning, image signing, GitOps promotion, admission checks, progressive rollout, and a declarative PostgreSQL request. It runs locally on kind and was also rebuilt and measured on Azure Kubernetes Service (AKS). The same versioned configuration describes both environments.

1.2 Problem and objectives

A valid image signature alone does not establish that the expected repository, branch, or workflow produced the image. Warden therefore checks the artifact digest and its build identity, while separating the pipeline from cluster credentials. The project set six measurable objectives:

Objective	Experiment
Create and deploy a service from one command	E1
Refuse artifacts that fail admission checks	E2
Abort a faulty canary without a manual rollback	E3
Provide a PostgreSQL database from a short request	E4
Measure delivery from deployment history	E5
Rebuild the environment from versioned configuration	E6

Table 1.1. Objectives and their evaluation experiments.

1.3 Cloud services and scope

AKS supplies managed Kubernetes; GitHub Actions runs the build; GitHub Container Registry stores immutable images; Microsoft Entra ID and GitHub OpenID Connect provide workload identity; Sigstore issues signing certificates and records transparency entries; Azure Storage holds encrypted OpenTofu state. Argo CD applies the GitOps repository, Argo Rollouts controls canary traffic, and Crossplane composes a CloudNativePG database. Together, these services cover compute, registry, identity, storage, infrastructure as code, and delivery automation.

The implemented scope includes enforceable digest, registry, and Pod hygiene rules, a tested cryptographic policy in Audit, and self-service databases through CloudNativePG. Azure Flexible Server support is implemented on review branches but still needs a live provisioning and lifecycle test. Chapter 3 distinguishes controlled Deny-mode tests from installed enforcement. The evaluation also excludes production availability, multiple regions, and runtime intrusion detection.

1.4 Research and technical foundations

The platform connects several established ideas. Borg illustrates declarative workload management and automated recovery in a shared cluster. The in-toto work treats software integrity as a property of the steps that produce an artifact. Sigstore makes artifact signing practical through identity-backed certificates, short-lived keys, and public transparency records. SLSA defines interoperable provenance for describing where, when, and how an artifact was built. Warden applies these ideas through Kubernetes orchestration and signed build provenance. [1], [2], [3], [4]

OpenGitOps describes desired state that is versioned, pulled automatically, and continuously reconciled. Google's canary-release guidance explains how partial deployments and observed behavior can inform a rollout decision. DORA provides a framework for measuring delivery throughput and instability. These sources supply the technical foundations for Warden's GitOps path, canary experiments, and delivery measurements. [5], [6], [7]

1.5 Project contribution

The engineering contribution is the integration and evaluation of the complete delivery path. The project supplies a service-creation CLI, a reusable build and signing workflow, constrained promotion, policy fixtures, experiment runners, and delivery-metric calculations. Its evaluation reaches beyond a successful deployment: it attempts invalid artifacts, injects faults, checks controller reconciliation, and recreates the environment. This makes both the working behavior and the remaining gaps inspectable.

CHAPTER 2: METHODOLOGY

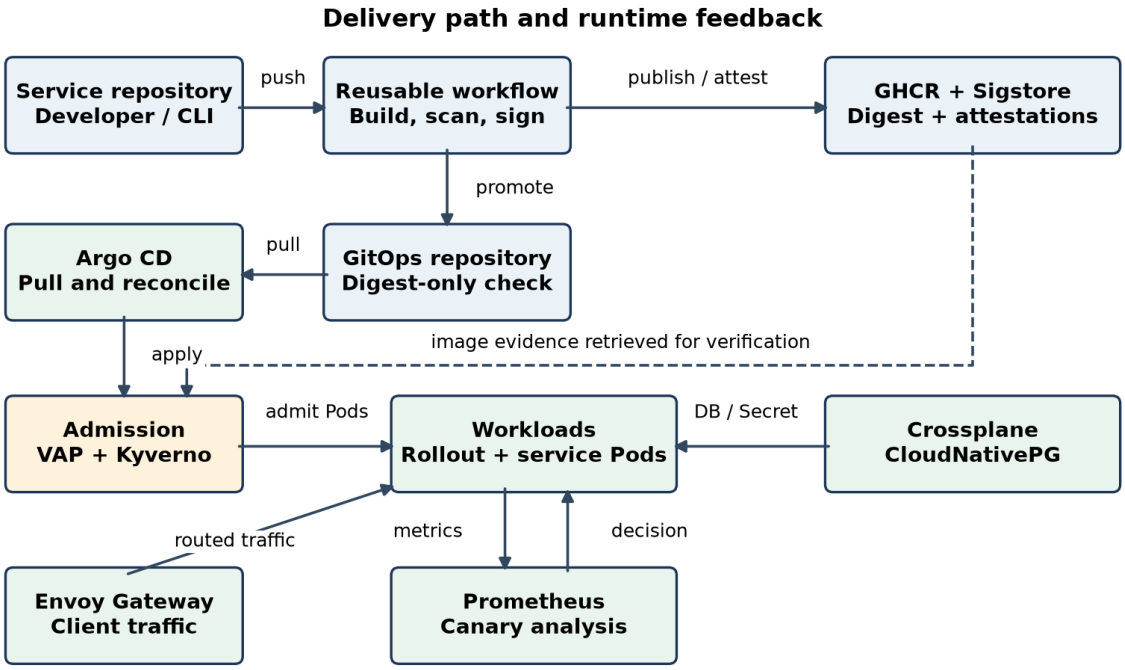


Figure 2.1. Warden delivery and cluster control flow.

2.1 Workspace and infrastructure

The project separates infrastructure, GitOps state, admission policies, and platform tooling into four repositories. OpenTofu prepares Azure resources. For local trials, a kind cluster replaces AKS while the GitOps and admission paths stay comparable. Pinned component versions and image digests keep rebuilds repeatable. Cluster creation and destruction were timed from a clean state, and failed attempts remain in the measurement records.

2.2 Service creation and trusted build

The Warden command creates a service repository, a small application, a reusable delivery workflow, deployment manifests, and a catalog entry. A push calls the shared platform workflow. It builds and scans the image, signs the digest, attaches provenance, and opens a narrow GitOps pull request that changes the deployed digest. The signing job alone receives an identity token. No delivery job receives cluster credentials; Argo CD later pulls the merged change. Third-party workflow actions are pinned to full commit hashes. The artifact identity and production steps follow software supply-chain practices described by in-toto and Sigstore [2], [3].

2.3 Promotion, admission, and rollout

A required digest-only check constrains each promotion pull request to one image digest. The promoter application can merge after that check passes. Argo CD applies the desired state. Native admission rules reject mutable tags, unapproved registries, and unsafe Pod settings. Kyverno verifies signatures and provenance against the intended build identity in Audit mode. For the controlled attack experiment, the same policy was exercised in Deny mode. Argo Rollouts then sends a small traffic share to the new version and uses Prometheus metrics to promote or abort it. Envoy Gateway supplies the route. Pull-based reconciliation and partial rollouts follow the GitOps and canary principles in [5] and [6].

2.3.1 Trust boundaries and control design

The repositories separate the code that builds an application from the configuration that approves and deploys it. A service repository cannot directly rewrite the policy repository through its promotion credential. The build job cannot request a signing identity token; that permission belongs only to the signing job, which does not check out project code. This separation reduces the credentials available to application build scripts. The GitOps check also rejects a changed image name or an unrelated configuration edit.

Risk	Implemented control	Evidence
Mutable or substituted image	Digest references and registry allowlist	A01, A06
Untrusted build identity	Signed provenance binds digest, workflow, caller repository, and branch	A02–A08; Audit live
Build code accesses signing identity	Identity token restricted to isolated signing job	A10
Promotion changes unrelated configuration	Required digest-only check with no bypass	A11
Out-of-band policy drift	Argo CD reconciliation restores desired state	A12

Table 2.1. Selected risks, controls, and their evaluation coverage. Cryptographic admission is staged.

2.4 Database and measurement

A namespaced database request is reconciled by Crossplane and CloudNativePG. The composition creates a predictable connection Secret, allowing the application manifest to remain the same across kind and AKS. Database readiness and a client connection were recorded. Delivery metrics use deployment events rather than a transient cluster log. Attack outcomes, canary timing, rebuilds, and verification latency were retained with their experiment records.

2.5 Progressive delivery under measured traffic

The rollout advances through 5%, 25%, and 50% canary traffic before full promotion. A synthetic load supplies requests while Prometheus evaluates success rate and latency. The experiments inject either HTTP errors or extra latency, so both reliability and response time are tested. An unsuccessful analysis aborts the canary; a subsequent corrected release still passes through the normal build and promotion path. The report distinguishes time to abort from time to deploy a complete fix, following the canary approach described in [6].

Traffic routing required coordination between two controllers. Argo CD maintains the declared configuration, while Argo Rollouts must change route weights during a canary. Without an explicit field-ownership exception, reconciliation reset those weights and invalidated the intended experiment. The final configuration permits Rollouts to manage the traffic weights while Argo CD continues to reconcile the surrounding resource.

2.6 Evaluation design and reproducibility

Eight experiments cover service creation, security controls, canary behavior, database provisioning, delivery history, rebuilds, comparison with another platform, and verification overhead. Each stores the observation alongside its procedure. Successful rebuilds are summarized separately from failed attempts; a failed attempt remains in the measurement records. Release metrics are cross-checked against the experiment runner so that an incorrect event count does not become a performance claim. Metric definitions follow DORA's five measures of delivery performance [6].

The environments answer different questions. kind makes repeated local trials affordable, while AKS tests cloud provisioning, reconciliation, and the policy path on managed Kubernetes. Results are labeled by environment because one successful local series does not establish identical cloud behavior. Small samples and deliberate fault injection limit the conclusions: the experiments demonstrate the recorded behavior, not a production reliability rate.

CHAPTER 3: RESULT ANALYSIS

3.1 Experiment summary

ID	Measured result	Interpretation
E1	5 full kind runs; median 8.6 min; 0 manual steps	One-command local path shown; Azure and hand-timed baseline open.
E2	15/15 security checks passed on each environment	Admission, workflow, promotion, and reconciliation controls; image policy audits live.
E3	App-only: 4/4 faults aborted; 12/12 healthy promoted	Median abort 147.6 s; faulty request share median 0.39%.
E4	Database ready: kind 804 s cold, 21 s warm; AKS 138 s	Same request and Secret shape; both backed by CloudNativePG.
E5	App-only batch: 20 deployments, 5 per service	Median lead time 12.5 min; failure and rework 20%; recovery 12.6 min.
E6	Local 11/12 rebuilds; Azure 5 successful rebuilds	Azure success median 22 min, with 4 failed attempts documented.
E7	One run each against idpbuilder	Warden used 8.8 GiB; idpbuilder 2.2 GiB and 8.5 min to first app.
E8	Warm verification 14–15 s per Pod	Similar on kind and AKS; cost is inside the verification path.

Table 3.1. Recorded results; the sample and scope are stated for each experiment.

The five complete service-creation runs finished below the 15-minute target. The resumed books run is excluded from this sample. The result demonstrates the automated local path, while the planned Azure series and hand-timed baseline remain open.

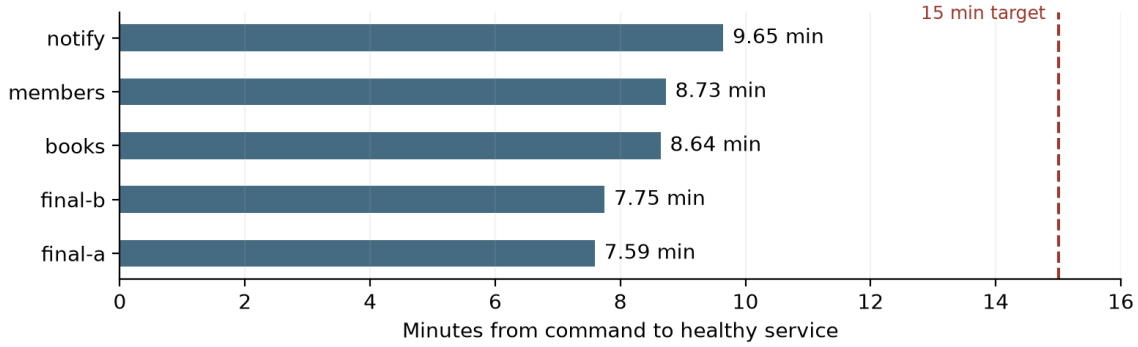


Figure 3.1. Complete local scaffolding trials, E1. Source: project experiment measurements.

3.2 Canary behavior

The controlled rollout series contained ten deliberately faulty and ten healthy releases. Every faulty release aborted, and every healthy release promoted. The median time from rollout start to abort was 147.2 seconds; the healthy promotion median was 471.6 seconds. The error trials exposed a median 0.397% of requests to the faulty canary, below the planned 0.5% share. The run data supports the rollback claim for kind; no comparable Azure canary series was run.

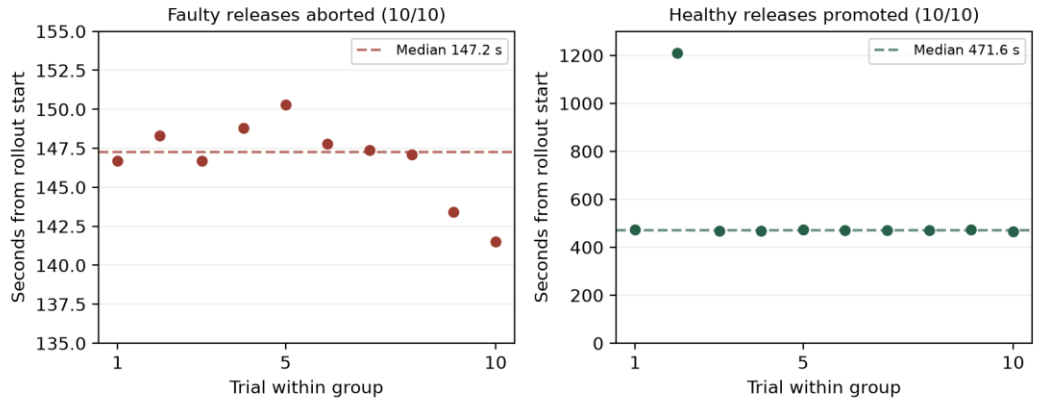


Figure 3.2. All 20 measured kind rollout outcomes. Panels use different vertical scales; dashed lines show medians. The healthy series includes the 1,209.5 s outlier.

The ten faulty trials comprise five error and five latency injections. In the error trials, the observed request failure share ranged from 0.325% to 0.509%; its median was 0.397%. The latency trials slowed a median 4.08% of service requests and produced no HTTP errors. These measures describe different failure modes and should not be combined into one success percentage.

A quick abort also differs from complete recovery. The median interval from abort to a corrected release being fully promoted was 759.2 seconds, including the new build and canary. A single healthy trial took 1,209.5 seconds to promote, while the other healthy trials took 466.2–474.1 seconds. Keeping that outlier visible shows the spread that a median alone would hide.

3.3 Delivery and rebuild evidence

The original 2026-10-26 E5 batch used owner merges after the digest-only check. Its 20% failure rate was planned and its lead time excluded human review. The separate 2026-10-27 app-only repeat records 20 deployments, five per service, with 16 successful and four injected-fault deployments. The promoter app merged every promotion after the required digest-only check, without an owner merge. Median change lead time was 12.5 minutes, change fail and rework rates were 20%, and median failed-deployment recovery was 12.6 minutes. The short synthetic batch does not describe normal team delivery frequency.

The canary records for the app-only repeat show 12 healthy promotions and four injected-error releases aborted by the success-rate analysis, each followed by a healthy fix. Fault aborts took a median 147.6 seconds, with 0.39% of requests failing during the canary at a 5% traffic share. Abort-to-fixed median was 756.6 seconds. One members CI capture timed out and a second healthy attempt overlapped the members fault test; both are retained as excluded records. The 20 deployments in the GitHub ledger agree with the measured set.

Service	Deployments	Median lead	Change fail	Median recovery
Books	5	12.8 min	20%	14.1 min
Loans	5	13.1 min	20%	11.8 min
Members	5	12.3 min	20%	12.6 min
Notify	5	12.5 min	20%	12.6 min

Table 3.2. Per-service DORA results in the 92-minute app-only experiment window.

The resulting deployment frequency, about 250 per day in this short test window, describes the scripted batch and is not a useful estimate of normal team cadence.

Service	GitOps pull request	Author and merger
notify	#97	warden-ccd-promoter
loans	#98	warden-ccd-promoter
members	#99	warden-ccd-promoter
books	#100	warden-ccd-promoter

Table 3.3. Verified v0.1.2 digest promotions on 27 October 2026.

Local rebuilds succeeded in 11 of 12 recorded runs, with a median of 14.9 minutes and a 24.2-minute 90th percentile. Five Azure rebuilds succeeded from an empty state with a median of 22 minutes. Four failed Azure attempts remain in the measurement records and were analyzed rather than discarded.

3.4 Admission and database limits

All 15 checks passed on kind and AKS, but they do not represent 15 admission rejections. The suite contains one admitted positive control, eight direct admission denials, one ReplicaSet case whose child Pods were refused, and five checks covering earlier prevention, credential absence, signing permissions, promotion constraints, and drift recovery. In the drift test, the policy was restored after 1.4 seconds on kind and 1.6 seconds on AKS.

This gives evidence for controls at several points in the delivery path. It does not establish fail-closed operation of the installed image policy, which remains in Audit. Warm verification added about 14–15 seconds per Pod, and transient registry or TUF fetch errors interrupted some legitimate checks. A new readiness run is required before enforcement changes.

The measured database request worked in both environments and generated a usable connection URI, with CloudNativePG selected on kind and AKS. Azure Flexible Server support is now implemented in draft infrastructure and GitOps pull requests but has not been merged or tested live. The result establishes portability of the request and Secret contract across clusters, not yet across database providers.

3.5 What the comparison establishes

The idpbuilder comparison helps explain Warden's resource cost. In one recorded run, idpbuilder reached GitOps and a first application in 8.5 minutes using 2.2 GiB. Warden's recorded platform footprint was 8.8 GiB and included admission checks, canary analysis, observability, and database infrastructure. The bootstrap paths differed: the baseline deployed a pre-built public image, while Warden's service-creation experiment included repository creation, building, signing, and promotion. The comparison describes scope and resource use; it is not an equal-feature speed benchmark.

3.6 Engineering findings

Several results depended on resolving interactions between components. The first promotion fix failed because the command-line client refused a blocked pull request before the server could apply the promoter's review bypass. Calling the merge endpoint with the expected head commit resolved that path; the digest-only check remained mandatory. The four later service releases provide direct evidence that the revised path worked.

Measurement code also needed validation. An early delivery-metric calculation counted only eight of twenty releases because older GitHub deployments became inactive and reconciliation announced a revision across multiple applications. Cross-checking the event ledger against the experiment runner exposed both errors. The corrected result counts twenty releases and four injected failures.

Database integration exposed another issue: operator-created Pods and initialization Jobs need narrowly scoped admission exceptions. A rule that looked acceptable in Audit could have blocked the database in Deny. Testing legitimate workload classes alongside attacks was necessary to judge the policy. The experiments also exposed controller and measurement problems that a successful deployment alone would not have found.

CHAPTER 4: CONCLUSION

Warden demonstrates a working Kubernetes delivery platform whose key behaviors can be traced to recorded experiments. It combines service scaffolding, an isolated signing workflow, digest promotion, admission controls, canary analysis, database requests, and repeatable infrastructure. The local service-creation median was 8.6 minutes; the app-only canary batch aborted all four injected faults and promoted all 12 measured healthy releases; and five Azure rebuilds recreated the platform with a median duration of about 22 minutes. These outcomes show an integrated system that was exercised beyond a single successful deployment.

Two promised outcomes need more work before they can be described as complete. The installed cryptographic admission policy still audits instead of denying, even though a controlled Deny-mode suite passed. Azure Flexible Server code is staged for review and still needs a live test. The E1 sample is also smaller than planned. These boundaries distinguish a tested feature from a production-ready path.

The next engineering steps are to remeasure image verification after a Kyverno fix, including legitimate Pods, registry failures, and timeout behavior before changing Audit to Deny; review and merge the Azure database changes, then test connection, deletion, and teardown; and expand service-creation and canary measurements on AKS. The report preserves what has been demonstrated and the evidence still needed for those claims.

TEAM RESPONSIBILITIES

The table records the project plan's responsibility allocation. Each member should confirm that the assigned area reflects work they personally completed before submission.

Member	Assigned area
Abhyuday Gupta	Platform architecture, trusted build workflow, CLI, integration, and evidence review
Aditya Chetan Thakur	Infrastructure, managed database path, and Azure rebuild evaluation
Sankalp Arya	Admission policies, threat model, and attack-suite evaluation
Shaurya Jain	GitOps delivery, canary rollout, and observability evaluation

Table 4.1. Planned work areas. Confirm the actual completed contributions with each member.

REFERENCES

[1] A. Verma, L. Pedrosa, M. R. Korupolu, D. Oppenheimer, E. Tune, and J. Wilkes, “Large-scale cluster management at Google with Borg,” in Proc. EuroSys, 2015. Available: https://research.google/pubs/large-scale-cluster-management-at-google-with-borg/

[2] S. Torres-Arias, H. Afzali, T. K. Kuppusamy, R. Curtmola, and J. Cappos, “in-toto: Providing farm-to-table guarantees for bits and bytes,” in Proc. 28th USENIX Security Symposium, 2019, pp. 1393–1410. Available: https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[3] Z. Newman, J. S. Meyers, and S. Torres-Arias, “Sigstore: Software Signing for Everybody,” in Proc. ACM SIGSAC Conference on Computer and Communications Security, 2022, pp. 2353–2367, doi: 10.1145/3548606.3560596.

[4] OpenSSF, “SLSA v1.0: Provenance.” Available: https://slsa.dev/spec/v1.0/provenance.

[5] OpenGitOps, “GitOps Principles,” version 1.0.0. Available: https://opengitops.dev/ (accessed Oct. 27, 2026).

[6] A. Warner and Š. Davidovič, “Canarying Releases,” in The Site Reliability Workbook, O’Reilly Media, 2018, ch. 16. Available: https://sre.google/workbook/canarying-releases/

[7] N. Harvey, “A history of DORA’s software delivery metrics,” DORA, Jan. 2, 2026. Available: https://dora.dev/insights/dora-metrics-history/ (accessed Oct. 27, 2026).